

CLEAN BOOK EDITION

Production React App 2026

A practical, project-based guide to building production-grade React applications with TypeScript, Next.js, TanStack Query, Zustand, Tailwind, shadcn/ui, forms, testing, and deployment.

How to Use This Book

This book teaches React through one running product: an issue tracker dashboard. Each part starts with the idea, then shows how it changes a real project. Keep the companion source bundle open while reading.

The book uses small examples first, then project code. The goal is not to memorize APIs; it is to learn how production React code is structured, reviewed, tested, and shipped.

LEARNING PATH

JS/TS foundation

↓

Thinking in React

↓

React core project

↓

Production architecture

↓

UI + forms + state/data

↓

Next.js + testing + deployment

Contents

How to Use This Book	2
Contents	3
Final Project	6
Part 1: JavaScript and TypeScript You Need for React	7
Data Transformation: Destructuring, Spread, and Arrays	8
Async API Functions and Modules	9
TypeScript Domain Types, Unions, and Generics	10
Narrowing UI States	11
React TypeScript Boundaries	11
Part 2: Thinking in React and Designing Components from UI/UX	12
How to Read a UI Like a React Engineer	12
UI State Model	13
Designing Component APIs	14
From Screen to Feature Folder	14
Part 3: React Core You Must Know	15
Project Setup: Vite Issue Board	15
JSX, Components, and Composition	16
Props and Component Boundaries	19
State and Rendering Behavior	20
Events and Controlled Inputs	21
Lists, Keys, and Conditional Rendering	22
Derived State	23
Effects: When to Use Them and When Not To	25
Refs and Escape Hatches	26
Custom Hooks	27
Part 4: React Best Practices and Techniques	28
State Ownership Rules	28
Component Design Patterns	29
Loading, Error, Empty, and Disabled States	30
Reusable UI API Design	30
Part 5: Architecture, Folder Structure, and Coding Standards	31
Production Folder Structure	31
Naming and Import Standards	32
API Layer Coding Standard	32
TypeScript and State Standards	33
Pull Request Checklist	33
Part 6: Build Real UI	34
Tailwind CSS for Production UI	34

shadcn/ui Component System	35
Dashboard Layout and Navigation	35
Accessibility and UI Polish	35
Storybook / Component Playground	36
Part 7: Forms and Validation	37
Manual Forms First	37
React Hook Form + Zod	37
Production Form UX	38
Part 8: Client State and Server State	39
The State Split	39
Zustand for Client UI State	40
TanStack Query for Server State	40
Mutations, Invalidation, and Optimistic Updates	41
URL State for Filters	41
Part 9: Data Tables and Dashboard Patterns	42
Table UX Basics	42
TanStack Table	42
Dashboard Metrics and Bulk Actions	43
Part 10: Next.js App Router	44
Why Next.js for Production React	44
App Router Structure	44
Server Components vs Client Components	45
Route Handlers and Server Actions	46
Authentication, Authorization, and Environment Config	46
Part 11: Common Tools Used in Production React Teams	47
Package Managers and Scripts	47
ESLint, Prettier, and TypeScript Strict	47
Useful Utility Libraries	48
Part 12: Backend Integration for a Frontend Engineer	49
Mock API First	49
DTOs and API Contracts	50
FastAPI, C#/.NET, or Next Route Handlers	50
Part 13: Testing and Quality	51
Testing Strategy	51
Vitest and React Testing Library	51
MSW for API Mocking	52
Playwright E2E Testing	52
Part 14: Production Hardening	53
Error Handling and Error Boundaries	53
Observability Basics	54
Performance Techniques	54
Security Basics	55

CI/CD and Dependency Maintenance	55
Part 15: Legacy React You Will See	56
Class Components	56
Old Redux Boilerplate	56
HOCs, Render Props, and Pages Router	57
Part 16: Capstone Project	58
Capstone Architecture	58
Build Checklist	59
Code Review and Portfolio README	59
Reference Basis	60

Final Project

The final project is a SaaS-style Issue Tracker Dashboard with authentication, projects, issues, filters, create/edit forms, validation, optimistic updates, role-based UI, tests, and deployment checklists.

MOCK UI - ISSUE BOARD MILESTONE

Issue Board

Project Alpha

Total

4

Open

2

Blocked

1

Done

1

Search issues...

Status: All

FINAL PROJECT ARCHITECTURE

Next.js App Router

↓ routes and layouts

features/issues

↓ API, components, hooks, schemas, types

TanStack Query

↓ server state cache

shadcn/ui + Tailwind

↓ production UI

React Hook Form + Zod

↓ forms and validation

Vitest + Testing Library + MSW + Playwright

↓ quality gate

PART 1

JavaScript and TypeScript You Need for React

Build a small Issue Data Playground before touching React. The goal is not to learn every JavaScript feature; it is to master the patterns that appear constantly inside React code.

Data Transformation: Destructuring, Spread, and Arrays

What is it? React components usually receive data, shape it, and render it. The everyday tools are destructuring, spread/rest, and array methods.

Why use it? These tools keep view code readable and prevent mutation bugs. Backend entities rarely match the exact shape your UI needs.

When to use it? Use these patterns when creating view models, updating state, filtering lists, or building dropdown options.

part-1-data-playground/issues.ts

```
const issue = {
  id: "issue_1",
  title: "Fix login loading state",
  status: "open",
  priority: "high",
  assignee: { name: "Ada" },
};

const { title, status, assignee } = issue;
const assigneeName = assignee?.name ?? "Unassigned";

const updatedIssue = { ...issue, status: "done" };

const visibleIssues = issues
  .filter((issue) => issue.status !== "done")
  .map(({ id, title, priority }) => ({
    id,
    label: `${priority.toUpperCase()} - ${title}`,
  }));
```

The destructuring line pulls out only the fields the UI needs. The spread update creates a new issue instead of mutating the existing one. The array chain filters domain data and maps it into UI-facing rows.

Do

```
const next = issues.map((issue) =>
  issue.id === id ? { ...issue, status: "done" }
  : issue
);
```

Don't

```
issue.status = "done";
issues.push(newIssue);
```

Tip

Name transformed values by business meaning: `visibleIssues` , `issueRows` , `assigneeOptions` . Avoid `data` , `result` , and `arr` .

Project step. In the Issue Data Playground, create `issues.ts` and write transformations for open issues, high-priority issues, and select options.

Async API Functions and Modules

What is it? Modules split code into files. Async functions let frontend code call APIs and return data.

Why use it? API code should not live inside random components. A clean API function can later be used by TanStack Query, Server Components, or tests.

```
// src/api/issues-api.ts
const API_BASE_URL = "https://jsonplaceholder.typicode.com";

export async function getIssues() {
  const response = await fetch(`${API_BASE_URL}/todos?_limit=5`);

  if (!response.ok) {
    throw new Error(`Failed to load issues: ${response.status}`);
  }

  const todos = await response.json();

  return todos.map((todo) => ({
    id: String(todo.id),
    title: todo.title,
    status: todo.completed ? "done" : "open",
  }));
}
```

`fetch` does not throw for every HTTP error, so checking `response.ok` is important. The API function also maps the external shape into the app shape.

Production habit

Keep URL construction, response parsing, and response mapping in the API layer. Components should not know raw backend field names unless the backend shape is intentionally the frontend shape.

API BOUNDARY

External API response
↓ map at boundary
App-friendly Issue[]
↓ used by
React components / query hooks

Task. Create `projects-api.ts` that maps JSONPlaceholder users into fake project objects.

TypeScript Domain Types, Unions, and Generics

What is it? TypeScript describes the shape of data. In React apps, the highest-value types are domain models, API responses, form values, props, and UI states.

Why use it? Most frontend bugs are shape bugs: nullable fields, wrong status strings, missing props, and mismatched request payloads.

`src/types/issue-types.ts`

```
export type IssueStatus = "open" | "in_progress" | "blocked" | "done";
export type IssuePriority = "low" | "medium" | "high";

export type Issue = {
  id: string;
  title: string;
  status: IssueStatus;
  priority: IssuePriority;
  assigneeName: string | null;
};

export type PaginatedResponse<T> = {
  items: T[];
  total: number;
  page: number;
  pageSize: number;
};
```

Unions prevent invalid status values. Generics describe reusable structures without losing the type inside them.

Do

```
type IssueStatus = "open" | "in_progress" |
  "blocked" | "done";
```

Don't

```
type IssueStatus = string;
type Issue = any;
```

Real-world warning

Using `any` to move faster usually moves bugs from the editor into runtime. Use `unknown` at unsafe boundaries and validate or narrow before using the value.

Narrowing UI States

What is it? Narrowing lets TypeScript understand which branch of a union you are currently handling.

Why use it? UI often has exact states: loading, error, empty, success, unauthorized. A discriminated union models that better than a pile of booleans.

```
type AsyncState<T> =  
  | { status: "loading" }  
  | { status: "error"; message: string }  
  | { status: "success"; data: T };  
  
function getIssueListLabel(state: AsyncState<Issue[]>) {  
  switch (state.status) {  
    case "loading":  
      return "Loading issues...";  
    case "error":  
      return `Could not load issues: ${state.message}`;  
    case "success":  
      return `${state.data.length} issues loaded`;  
  }  
}
```

Inside each `case`, TypeScript knows exactly which fields are available.

Tip

A single status field often produces clearer UI logic than `isLoading`, `isError`, `hasData`, and `isEmpty` flags that can contradict each other.

React TypeScript Boundaries

What is it? React TypeScript is mostly about typing component boundaries: props, callback props, children, events, and state.

```
type SearchInputProps = {  
  value: string;  
  onChange: (value: string) => void;  
};  
  
export function SearchInput({ value, onChange }: SearchInputProps) {  
  return (  
    <input  
      value={value}  
      onChange={(event) => onChange(event.target.value)}  
      placeholder="Search issues..."  
    />  
  );  
}
```

This component hides the DOM event and exposes a clean string callback. The parent does not need to know about `event.target.value`.

Code review question

Can another engineer use this component correctly by reading only its prop type? If not, the component API is probably unclear.

PART 2

Thinking in React and Designing Components from UI/UX

How to Read a UI Like a React Engineer

What is it? Thinking in React means turning a visual design into components, state, props, and data flow.

Why use it? Many engineers can write hooks, but still struggle to design clean components from a real screen.

MOCK UI - ISSUE BOARD MILESTONE

Issue Board

Project Alpha

Total

4

Open

2

Blocked

1

Done

1

Search issues...

Status: All

Start with regions: page shell, toolbar, summary cards, list/table, dialogs. Then ask which parts repeat and which parts own behavior.

Dashboard chart overflows on mobile

FROM DESIGN TO COMPONENT TREE

```
IssueBoardPage
├── BoardSummary
├── IssueToolbar
│   ├── SearchInput
│   └── StatusFilter
└── IssueList
    ├── EmptyState
    └── IssueCard
```

Tip

Name components from the product language first: IssueCard , IssueFilters , ProjectHeader .
Extract generic UI components later when repeated patterns are obvious.

UI State Model

What is it? A state model lists the values that can change the screen.

Why use it? It prevents you from putting every value in one global store.

ISSUE LIST STATE MODEL

Server state:

- issues
- projects
- users

Local UI state:

- selected issue
- dialog open/closed

URL state:

- search
- status filter
- page number

Form state:

- create/edit issue values

Client UI state:

- sidebar collapsed
- active modal

Decision rule

Start local. Move state only when there is a reason: shareability, server ownership, form behavior, or cross-feature UI state.

Designing Component APIs

What is it? A component API is the props contract. It is how other engineers use your component.

```
<IssueStatusBadge status="blocked" />

<ConfirmDialog
  title="Delete issue?"
  description="This action cannot be undone."
  onConfirm={handleDelete}
/>

<IssueTable
  issues={issues}
  onIssueSelect={handleIssueSelect}
  onStatusChange={handleStatusChange}
/>
```

Good component APIs use domain names, not vague flags. They expose intent, not implementation details.

Do

```
onStatusChange(issueId, "done")
onProjectArchive(projectId)
```

Don't

```
callback(data)
doThing(value)
flag1={true} flag2={false}
```

From Screen to Feature Folder

What is it? A feature folder groups code by business domain.

Why use it? Teams work on features, not file types. Keeping issue code near other issue code is easier than spreading it across global folders.

FEATURE FOLDER FOR ISSUES

```
src/features/issues/
  api/
    issues-api.ts
  components/
    issue-table.tsx
    issue-form.tsx
    issue-status-badge.tsx
  hooks/
    use-issues.ts
    use-update-issue.ts
  schemas/
    issue-schema.ts
  types/
    issue-types.ts
```

Code review question

If this file changes only when the Issues feature changes, why is it outside features/issues ?

PART 3

React Core You Must Know

Build the Vite Issue Board. This is the bridge from isolated React syntax to a working project.

Project Setup: Vite Issue Board

We use Vite here because it keeps React core visible. Next.js enters later when routing, layouts, Server Components, and production deployment matter.

terminal

```
npm create vite@latest issue-board-vite -- --template react-ts
cd issue-board-vite
npm install
npm run dev
```

Open <http://localhost:5173/> . Then clean the starter app.

src/App.tsx

```
export default function App() {
  return <h1>Issue Board</h1>;
}
```

terminal

```
mkdir src/components src/data src/hooks src/types
```

PART 3 FINAL STRUCTURE

```
issue-board-vite/
  src/
    App.tsx
    App.css
    components/
      board-summary.tsx
      empty-state.tsx
      issue-card.tsx
      issue-list.tsx
      search-input.tsx
      status-filter.tsx
    data/
      mock-issues.ts
    hooks/
      use-issue-board.ts
    types/
      issue-types.ts
```

JSX, Components, and Composition

What is it? JSX describes UI inside TypeScript. A component is a function that returns JSX. Composition means building a screen from smaller components.

Why use it? Components stop one screen from becoming a giant unreadable file. They create names for meaningful UI pieces.

When to use it? Extract a component when UI repeats, a section has one responsibility, or the design already gives the section a name.

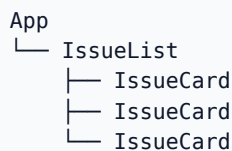
```
type UserCardProps = {
  name: string;
  role: string;
};

function UserCard({ name, role }: UserCardProps) {
  return (
    <article>
      <h2>{name}</h2>
      <p>{role}</p>
    </article>
  );
}

export default function App() {
  return (
    <main>
      <h1>Team</h1>
      <UserCard name="Ada" role="Frontend Lead" />
      <UserCard name="Grace" role="Backend Engineer" />
    </main>
  );
}
```

UserCard is reusable because it receives data through props. The parent composes the page by using it multiple times.

FIGURE 3.1 - COMPONENT TREE



Apply it to the project by creating the issue model, mock data, an issue card, and an issue list.

src/types/issue-types.ts

```
export type IssueStatus = "open" | "in_progress" | "blocked" | "done";
export type IssuePriority = "low" | "medium" | "high";

export type Issue = {
  id: string;
  title: string;
  description: string;
  status: IssueStatus;
  priority: IssuePriority;
  assigneeName: string | null;
  createdAt: string;
};
```


src/data/mock-issues.ts

```
import type { Issue } from "../types/issue-types";

export const mockIssues: Issue[] = [
  {
    id: "issue_1",
    title: "Login button does not show loading state",
    description: "The login form submits, but the button stays active.",
    status: "open",
    priority: "high",
    assigneeName: "Ada",
    createdAt: "2026-06-23",
  },
  {
    id: "issue_2",
    title: "Dashboard chart overflows on mobile",
    description: "The chart breaks the layout below tablet width.",
    status: "in_progress",
    priority: "medium",
    assigneeName: null,
    createdAt: "2026-06-22",
  },
];
```

src/components/issue-card.tsx

```
import type { Issue } from "../types/issue-types";

type IssueCardProps = {
  issue: Issue;
};

export function IssueCard({ issue }: IssueCardProps) {
  const assigneeLabel = issue.assigneeName ?? "Unassigned";

  return (
    <article className="issue-card">
      <div className="issue-card__header">
        <h3>{issue.title}</h3>
        <span className="issue-card__priority">{issue.priority}</span>
      </div>

      <p>{issue.description}</p>

      <dl className="issue-card__meta">
        <div><dt>Status</dt><dd>{issue.status}</dd></div>
        <div><dt>Assignee</dt><dd>{assigneeLabel}</dd></div>
        <div><dt>Created</dt><dd>{issue.createdAt}</dd></div>
      </dl>
    </article>
  );
}
```

src/components/issue-list.tsx

```
import type { Issue } from "../types/issue-types";
import { IssueCard } from "../issue-card";

type IssueListProps = {
  issues: Issue[];
};

export function IssueList({ issues }: IssueListProps) {
  return (
    <section className="issue-section">
      <div className="section-heading">
        <h2>Issues</h2>
        <p>{issues.length} issues in this project</p>
      </div>
    </section>
  );
}
```

src/components/issue-list.tsx (continued)

```
    </div>

    <div className="issue-list">
      {issues.map((issue) => (
        <IssueCard key={issue.id} issue={issue} />
      ))}
    </div>
  </section>
);
}
```

Tip

Start clear before generic. IssueCard is better than UniversalCard at this stage because the domain is obvious.

Do

```
<IssueList issues={mockIssues} />
```

Don't

```
// hundreds of lines of issue card JSX inside
App.tsx
```

Checkpoint. Run `npm run dev`. You should see a static Issue Board with issue cards.

Props and Component Boundaries

What is it? Props pass data down. Callback props let children report user actions up.

Why use it? A child component should not secretly change parent data. It should report user intent and let the parent decide what to do.

```
type StatusButtonProps = {
  label: string;
  nextStatus: string;
  onStatusChange: (status: string) => void;
};

function StatusButton({ label, nextStatus, onStatusChange }: StatusButtonProps) {
  return <button onClick={() => onStatusChange(nextStatus)}>{label}</button>;
}
```

FIGURE 3.2 - DATA DOWN, EVENTS UP

App passes issues and onStatusChange

↓

IssueList passes one issue and onStatusChange

↓

IssueCard renders buttons

↑

User clicks Complete

↑

IssueCard calls onStatusChange(issue.id, "done")

↑

FIGURE 3.2 - DATA DOWN, EVENTS UP (CONTINUED)

App handles the request

src/components/issue-card.tsx

```
type IssueCardProps = {
  issue: Issue;
  onStatusChange: (issueId: string, status: IssueStatus) => void;
};

<button onClick={() => onStatusChange(issue.id, "in_progress")}>Start</button>
<button onClick={() => onStatusChange(issue.id, "blocked")}>Block</button>
<button onClick={() => onStatusChange(issue.id, "done")}>Complete</button>
```

src/App.tsx

```
export default function App() {
  function handleStatusChange(issueId: string, status: IssueStatus) {
    console.log("Status change requested:", { issueId, status });
  }

  return <IssueList issues={mockIssues} onStatusChange={handleStatusChange} />;
}
```

Production habit

Name callback props by product intent: `onStatusChange` , `onProjectArchive` , `onIssueSelect` . Avoid `callback` and `doThing` .

Checkpoint. Clicking a status button logs the issue ID and next status.

State and Rendering Behavior

What is it? State is component memory. When state changes, React runs the component again and updates the UI.

When to use it? Use state for values that change over time and affect the screen: search text, selected issue, dialog open state, or locally edited issue data.

```
const [count, setCount] = useState(0);

<button onClick={() => setCount((current) => current + 1)}>
  Count: {count}
</button>
```

Use a functional update when the next value depends on the previous value.

src/App.tsx

```
const [issues, setIssues] = useState<Issue[]>(mockIssues);

function handleStatusChange(issueId: string, status: IssueStatus) {
  setIssues((currentIssues) =>
    currentIssues.map((issue) =>
      issue.id === issueId ? { ...issue, status } : issue
    )
  );
}
```

FIGURE 3.3 - STATE UPDATE PATH

User clicks Complete
↓
IssueCard calls onStatusChange
↓
App creates next issues array
↓
React renders updated IssueCard

Do

```
setIssues((currentIssues) =>
  currentIssues.map((issue) =>
    issue.id === issueId ? { ...issue, status } :
    issue
  )
);
```

Don't

```
issue.status = "done";
setIssues(issues);
```

Debug checklist

If the UI does not update, check mutation first. Did you create a new array/object, or did you change the old one?

Task. Disable the Complete button when the issue is already **done** .

Events and Controlled Inputs

What is it? A controlled input is an input whose value is owned by React state.

Why use it? Controlled inputs make filters and forms predictable.

```
const [search, setSearch] = useState("");

<input
  value={search}
  onChange={(event) => setSearch(event.target.value)}
  placeholder="Search..."
/>
```

FIGURE 3.4 - CONTROLLED INPUT LOOP

React state: search
↓
input value
↓ user types
onChange event
↓
setSearch(nextValue)
↓
React renders again

src/components/search-input.tsx

```
type SearchInputProps = {
  value: string;
  onChange: (value: string) => void;
};

export function SearchInput({ value, onChange }: SearchInputProps) {
  return (
    <label className="search-input">
      <span>Search issues</span>
      <input
        value={value}
        onChange={(event) => onChange(event.target.value)}
        placeholder="Search by title or description..."
      />
    </label>
  );
}
```

src/App.tsx

```
const [search, setSearch] = useState("");
const normalizedSearch = search.trim().toLowerCase();

const visibleIssues = issues.filter((issue) => {
  if (normalizedSearch.length === 0) return true;

  return (
    issue.title.toLowerCase().includes(normalizedSearch) ||
    issue.description.toLowerCase().includes(normalizedSearch)
  );
});
```

Tip

Reusable inputs should usually expose clean values, not raw DOM events. `onChange(value)` is easier for parents than `onChange(event)`.

Checkpoint. Typing in the search box filters issues by title or description.

Lists, Keys, and Conditional Rendering

What is it? List rendering turns arrays into UI. Conditional rendering shows different UI depending on data or state.

Why use it? Real apps render lists everywhere: table rows, cards, dropdown options, notifications, and menu items.

```
function ProductList({ products }: { products: Product[] }) {
  if (products.length === 0) {
    return <p>No products found.</p>;
  }

  return (
    <ul>
      {products.map((product) => (
        <li key={product.id}>{product.name}</li>
      ))}
    </ul>
  );
}
```

src/components/empty-state.tsx

```
export function EmptyState({ title, description, action }: EmptyStateProps) {
  return (
    <div className="empty-state">
      <h3>{title}</h3>
      <p>{description}</p>
      {action}
    </div>
  );
}
```

src/components/issue-list.tsx

```
if (issues.length === 0) {
  return (
    <EmptyState
      title="No issues found"
      description="Try changing your search text or clearing filters."
      action={emptyAction}
    />
  );
}

return issues.map((issue) => (
  <IssueCard key={issue.id} issue={issue} onStatusChange={onStatusChange} />
));
```

Do

```
<IssueCard key={issue.id} issue={issue} />
```

Don't

```
<IssueCard key={index} issue={issue} />
```

Real-world warning

An empty area makes users think the app is broken. A useful empty state explains what happened and gives the next action.

Derived State

What is it? Derived state is data calculated from existing state or props.

Why use it? It avoids duplicated values that can get out of sync.

When to use it? Use it for visible rows, counts, selected entity, totals, badges, and dashboard metrics.

```
const visibleIssues = issues.filter((issue) =>
  issue.title.toLowerCase().includes(search.toLowerCase())
);

const openCount = issues.filter((issue) => issue.status === "open").length;
const doneCount = issues.filter((issue) => issue.status === "done").length;
```

src/components/board-summary.tsx

```
type BoardSummaryProps = {
  total: number;
  open: number;
  inProgress: number;
  blocked: number;
  done: number;
  visible: number;
};

export function BoardSummary(props: BoardSummaryProps) {
  return (
    <section className="board-summary" aria-label="Issue board summary">
      <div><span>Total</span><strong>{props.total}</strong></div>
      <div><span>Open</span><strong>{props.open}</strong></div>
      <div><span>In progress</span><strong>{props.inProgress}</strong></div>
      <div><span>Blocked</span><strong>{props.blocked}</strong></div>
      <div><span>Done</span><strong>{props.done}</strong></div>
      <div><span>Visible</span><strong>{props.visible}</strong></div>
    </section>
  );
}
```

FIGURE 3.5 - SOURCE DATA AND DERIVED VIEW DATA

Source state:
issues + search

↓ calculate

Derived data:
visibleIssues
openCount
doneCount
blockedCount

↓ render

UI:
IssueList
BoardSummary

Tip

When you want to create a new state variable, ask: Can I calculate this from existing state or props?
If yes, calculate it.

Do

```
const openCount = issues.filter((issue) =>
  issue.status === "open").length;
```

Don't

```
const [openCount, setOpenCount] = useState(0);
```

Task. Add a `highPriority` number to the summary.

Effects: When to Use Them and When Not To

What is it? An Effect lets React synchronize with something outside React.

When to use it? Use Effects for document title, local storage, timers, subscriptions, analytics, or non-React widgets.

When not to use it? Do not use Effects for normal filtering, sorting, counting, or formatting.

Do

```
useEffect(() => {
  document.title = `${issueSummary.visible}
    visible issues`;
}, [issueSummary.visible]);
```

Don't

```
useEffect(() => {
  setVisibleIssues(issues.filter(...));
}, [issues, search]);
```

src/App.tsx

```
const [search, setSearch] = useState(() => {
  return localStorage.getItem("issue-board-search") ?? "";
});

useEffect(() => {
  document.title = `${issueSummary.visible} visible issues`;
}, [issueSummary.visible]);

useEffect(() => {
  localStorage.setItem("issue-board-search", search);
}, [search]);
```

FIGURE 3.6 - EFFECT BOUNDARY

React state
search
issueSummary.visible

↓ useEffect

External systems
localStorage
document.title

Real-world warning

A lot of messy React code comes from unnecessary Effects. Before writing one, ask: What external system am I synchronizing with?

Checkpoint. Search text persists after refresh, and the browser title shows the visible issue count.

Refs and Escape Hatches

What is it? A ref stores a value that persists between renders without causing a re-render. Refs are often used for DOM access.

Why use it? Some browser actions are imperative: focus an input, scroll to an element, measure a DOM node.

```
const inputRef = useRef<HTMLInputElement | null>(null);

<input ref={inputRef} />
<button onClick={() => inputRef.current?.focus()}>Focus input</button>
```

src/components/search-input.tsx

```
import type { Ref } from "react";

type SearchInputProps = {
  value: string;
  onChange: (value: string) => void;
  inputRef?: Ref<HTMLInputElement>;
};

export function SearchInput({ value, onChange, inputRef }: SearchInputProps) {
  return (
    <label className="search-input">
      <span>Search issues</span>
      <input
        ref={inputRef}
        value={value}
        onChange={(event) => onChange(event.target.value)}
        placeholder="Search by title or description..."
      />
    </label>
  );
}
```

src/App.tsx

```
const searchInputRef = useRef<HTMLInputElement | null>(null);

function clearSearch() {
  setSearch("");
  localStorage.removeItem("issue-board-search");
  searchInputRef.current?.focus();
}
```

Tip

If changing a value should update the screen, use state. If the value is only needed for an imperative action like focus, a ref may be right.

Do

```
searchInputRef.current?.focus();
```

Don't

```
searchRef.current = nextSearch; // hidden UI  
state; screen will not re-render
```

Custom Hooks

What is it? A custom hook is a function that uses React hooks and extracts reusable component logic. Hook names start with `use`.

Why use it? Components should read like screens, not piles of filtering, counting, and state workflow code.

```
function useCounter(initialValue = 0) {  
  const [count, setCount] = useState(initialValue);  
  
  return {  
    count,  
    increase: () => setCount((current) => current + 1),  
    reset: () => setCount(initialValue),  
  };  
}
```

`src/hooks/use-issue-board.ts`

```
export type IssueStatusFilter = "all" | IssueStatus;  
  
type UseIssueBoardInput = {  
  issues: Issue[];  
  search: string;  
  statusFilter: IssueStatusFilter;  
};  
  
export function useIssueBoard({ issues, search, statusFilter }: UseIssueBoardInput) {  
  const normalizedSearch = search.trim().toLowerCase();  
  
  const visibleIssues = useMemo(() => {  
    return issues.filter((issue) => {  
      const matchesSearch =  
        normalizedSearch.length === 0 ||  
        issue.title.toLowerCase().includes(normalizedSearch) ||  
        issue.description.toLowerCase().includes(normalizedSearch);  
  
      const matchesStatus = statusFilter === "all" || issue.status === statusFilter;  
  
      return matchesSearch && matchesStatus;  
    });  
  }, [issues, normalizedSearch, statusFilter]);  
  
  const summary = useMemo(() => ({  
    total: issues.length,  
    open: issues.filter((issue) => issue.status === "open").length,
```

src/hooks/use-issue-board.ts (continued)

```
    inProgress: issues.filter((issue) => issue.status === "in_progress").length,  
    blocked: issues.filter((issue) => issue.status === "blocked").length,  
    done: issues.filter((issue) => issue.status === "done").length,  
    visible: visibleIssues.length,  
  }), [issues, visibleIssues.length]);  
  
  return { visibleIssues, summary };  
}
```

src/App.tsx

```
const [statusFilter, setStatusFilter] = useState<IssueStatusFilter>("all");  
  
const { visibleIssues, summary } = useIssueBoard({  
  issues,  
  search,  
  statusFilter,  
});
```

FIGURE 3.7 - CUSTOM HOOK AS VIEW-MODEL LOGIC

App state:
issues
search
statusFilter

↓

useIssueBoard()

↓

visibleIssues
summary

↓

BoardSummary
IssueList

Production habit

Feature-specific hooks should live near the feature. Later this moves to `src/features/issues/hooks/use-issue-board.ts`.

Do

```
const { visibleIssues, summary } =  
  useIssueBoard({ issues, search, statusFilter  
});
```

Don't

```
const data = useThing(issues, search, true,  
  false);
```

Task. Add a priority filter using the same pattern as `StatusFilter`.

PART 4

React Best Practices and Techniques

State Ownership Rules

What is it? State ownership means choosing the closest component or tool that should own a value.

Why use it? It prevents prop drilling, duplicated state, and global-store sprawl.

When to use it? Use local state first; lift state only when multiple components need it; move server data to TanStack Query later.

```
// Good: local UI state in a component
const [isCreateDialogOpen, setCreateDialogOpen] = useState(false);

// Good: derived data calculated from source state
const visibleIssues = issues.filter(matchesFilters);
```

Apply it to the project. In the Vite board, `App` owns issues, search, and status filter. Later, server-owned issues move to TanStack Query, while dialog open state can stay local or in Zustand.

Tip

If only one component uses a value, that component probably owns it.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

Component Design Patterns

What is it? Component patterns are recurring ways to structure UI APIs.

Why use it? They keep components flexible without making them too generic.

When to use it? Use controlled components for values owned by parents; use composition for layout; use compound components when children are tightly related.

```
<Dialog open={open} onOpenChange={setOpen}>
  <DialogTrigger>Create issue</DialogTrigger>
  <DialogContent>
    <IssueForm onSubmit={handleSubmit} />
  </DialogContent>
</Dialog>
```

Apply it to the project. The final app uses controlled dialogs, composition-based cards, and domain components like `IssueStatusBadge`.

Don't

Do not create a universal mega-component that handles every business case. Clear domain components are easier to maintain.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

Loading, Error, Empty, and Disabled States

What is it? Production UI states are the visual answers for non-happy paths.

Why use it? Users need to understand what the app is doing, what went wrong, and what they can do next.

When to use it? Add these states whenever a component depends on data, permissions, async actions, or filters.

```
if (isLoading) return <IssueTableSkeleton />;
if (error) return <ErrorState title="Could not load issues" onRetry={refetch} />;
if (issues.length === 0) return <EmptyState title="No issues found" />;
```

Apply it to the project. The Vite board already has an empty state. Later, TanStack Query adds loading/error states around server data.

Tip

Write empty-state copy like product UI, not debug logs. "No issues found" is better than "No data".

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

Reusable UI API Design

What is it? A reusable UI API is the prop contract for shared components.

Why use it? Good props make the component easy to use without reading its internals.

When to use it? Use clear variants, domain callbacks, and `children` for flexible content.

```
<Button variant="destructive" onClick={handleDelete}>Delete</Button>
<EmptyState title="No issues" description="Create the first issue." action={<Button>Create</Button>} />
```

Apply it to the project. In the production project, shared UI components live in `components/ui` and business components live in `features/*/components` .

Tip

If a shared component needs many product-specific props, it is probably not truly shared.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

PART 5

Architecture, Folder Structure, and Coding Standards

Turn the project from a learning app into a codebase a team can maintain.

Production Folder Structure

What is it? Folder structure is the map of responsibilities in your codebase.

Why use it? It tells engineers where code belongs and reduces merge conflicts and duplicated logic.

RECOMMENDED STRUCTURE

```
src/  
  app/  
    (auth)/login/page.tsx  
    (dashboard)/layout.tsx  
    (dashboard)/issues/page.tsx  
    api/health/route.ts  
  components/  
    ui/  
    layout/  
    feedback/  
  features/  
    auth/  
    projects/  
    issues/  
  hooks/  
  lib/  
  stores/  
  types/  
  tests/
```

Folder rule

If code belongs only to Issues, keep it under features/issues . Move it to shared folders only after multiple features need it.

Naming and Import Standards

What is it? Naming standards make files predictable and search-friendly.

Why use it? Consistent names speed up code review and onboarding.

When to use it? Use kebab-case for files, PascalCase for components, camelCase for functions, and `use` prefix for hooks.

```
// Files  
issue-table.tsx  
use-issues.ts  
issues-api.ts  
issue-schema.ts  
  
// Components and hooks  
export function IssueTable() {}
```

continued

```
export function useIssues() {}
```

Tip

Prefer named exports for app code. Use default exports where Next.js expects them, such as `page.tsx`.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

API Layer Coding Standard

What is it? The API layer hides HTTP details behind typed functions.

Why use it? It prevents URLs, headers, and response mapping from spreading through UI components.

When to use it? Use it for every backend interaction.

```
export async function getIssues(filters: IssueFilters): Promise<Issue[]> {  
  const response = await apiClient.get("/issues", { searchParams: filters });  
  return issueListResponseSchema.parse(response);  
}
```

Apply it to the project. TanStack Query should call API functions, not inline fetch blocks inside components.

Don't

Do not show toast messages inside API functions. Return errors; let the UI decide how to communicate them.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

TypeScript and State Standards

What is it? Coding standards make type and state choices consistent across the team.

Why use it? They stop every engineer from inventing a different style.

When to use it? Apply them to props, API types, form schemas, stores, and query hooks.


```
// State decision rule
Local UI state    -> useState
Form state        -> React Hook Form
Server state      -> TanStack Query
Client UI state   -> Zustand
Shareable filters -> URL search params
```

Tip

Use TypeScript strict mode. Avoid any . Use unknown at unsafe boundaries and validate with Zod when data crosses a runtime boundary.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

Pull Request Checklist

What is it? A PR checklist is a quality gate before merging.

Why use it? It catches missing states, leaked secrets, accessibility issues, and wrong state ownership.

When to use it? Use it for every feature PR.

- Does TypeScript pass?
- Does lint pass?
- Is API logic outside UI?
- Are loading, error, and empty states handled?
- Are forms validated?
- Are important flows tested?
- Are permissions enforced on the server?
- Are secrets kept out of client code?

Tip

A good PR should be understandable by reading file names, component props, and tests before diving into implementation details.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

PART 6

Build Real UI

Build the visual system for the production app using Tailwind, shadcn/ui, accessible primitives, and repeatable UI states.

Tailwind CSS for Production UI

What is it? Tailwind is a utility-first CSS framework: you style elements by combining small utility classes.

Why use it? It keeps styling close to markup and reduces custom CSS drift.

When to use it? Use it for layout, spacing, typography, responsive behavior, and state styling.

```
<section className="rounded-xl border bg-white p-6 shadow-sm">
  <h2 className="text-xl font-semibold tracking-tight">Issues</h2>
  <p className="text-sm text-muted-foreground">Recent project activity</p>
</section>
```

Apply it to the project. Replace the manual CSS from the Vite board with Tailwind utilities when the Next.js project begins.

Tip

Do not put business logic into class strings. Calculate business state first, then choose classes.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

shadcn/ui Component System

What is it? shadcn/ui gives copy-in components built on accessible primitives and Tailwind.

Why use it? Because the code lives in your repo, you can customize it instead of fighting a black-box package.

When to use it? Use it for buttons, dialogs, forms, tables, dropdowns, cards, and toasts.

```
npx shadcn@latest init
npx shadcn@latest add button card dialog input form table dropdown-menu sonner
```

Apply it to the project. The production app will place shadcn components under `components/ui`. Domain components wrap them with issue/project-specific behavior.

Tip

Treat shadcn/ui as a starting point for your design system, not a vendor component library you never touch.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

Dashboard Layout and Navigation

What is it? A dashboard layout defines the shell around feature pages.

Why use it? Users need consistent navigation, page hierarchy, and responsive behavior.

When to use it? Use it for authenticated sections such as dashboard, projects, and issues.

```
export function AppShell({ children }: { children: React.ReactNode }) {
  return (
    <div className="grid min-h-screen grid-cols-[260px_1fr]">
      <Sidebar />
      <main className="p-6">{children}</main>
    </div>
  );
}
```

Tip

Keep layout components free of feature business logic. A sidebar can know navigation links; it should not know how issues are fetched.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

Accessibility and UI Polish

What is it? Accessibility means the UI works with keyboard, screen readers, and clear semantics.

Why use it? It improves usability and avoids hidden defects that tests often miss.

When to use it? Use semantic HTML, labels, focus states, accessible dialogs, and meaningful button text.

```
<button type="button" aria-label="Archive project">
  <ArchiveIcon aria-hidden="true" />
</button>

<label htmlFor="issue-title">Title</label>
<input id="issue-title" />
```

Tip

Prefer buttons over clickable divs. If it acts like a button, make it a button.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

Storybook / Component Playground

What is it? A component playground lets you build and review UI states in isolation.

Why use it? It catches visual states that are hard to reach manually in the app.

When to use it? Use it for reusable UI, complex forms, table states, empty states, and design-system components.

```
export const Empty = {
  args: {
    issues: [],
    isLoading: false,
  },
};

export const Loading = {
  args: {
    issues: [],
    isLoading: true,
  },
};
```

Tip

Storybook is valuable for reusable UI. It may be overkill for a small app with only a few screens.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

PART 7

Forms and Validation

Build create and edit workflows with user-friendly validation and reliable submit behavior.

Manual Forms First

What is it? A manual form uses controlled inputs and submit handlers without a form library.

Why use it? It shows why production forms need a better abstraction.

When to use it? Use it only for small forms or learning.

```
const [title, setTitle] = useState("");

<form onSubmit={(event) => {
  event.preventDefault();
  createIssue({ title });
}}>
  <input value={title} onChange={(event) => setTitle(event.target.value)} />
</form>
```

Tip

Once you have validation, default values, edit forms, server errors, and field errors, move to React Hook Form.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

React Hook Form + Zod

What is it? React Hook Form manages form state. Zod validates runtime data and can infer TypeScript types.

Why use it? Together they keep form values, validation, and request payloads consistent.

When to use it? Use them for production create/edit forms.

```
const issueSchema = z.object({
  title: z.string().min(3),
  priority: z.enum(["low", "medium", "high"]),
});

type IssueFormValues = z.infer<typeof issueSchema>;

const form = useForm<IssueFormValues>({
  resolver: zodResolver(issueSchema),
  defaultValues: { title: "", priority: "medium" },
});
```

Apply it to the project. The create issue and edit issue dialogs share `IssueForm` with different default values and submit handlers.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

Production Form UX

What is it? Form UX is how users experience validation, saving, errors, and success.

Why use it? A technically correct form can still feel broken if it lacks feedback.

When to use it? Use loading states, field errors, server errors, dirty state, reset behavior, and toasts.

```
<Button type="submit" disabled={form.formState.isSubmitting}>
  {form.formState.isSubmitting ? "Saving..." : "Save issue"}
</Button>

{serverError ? <p role="alert">{serverError}</p> : null}
```

Tip

Disable submit while saving, but do not hide errors. Users need to know whether the form failed validation or the server rejected it.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

PART 8

Client State and Server State

Replace local learning-state with production data flow: server state, client UI state, form state, and URL state.

The State Split

What is it? State split means using the right state tool for each kind of state.

Why use it? It prevents Zustand, Context, or useState from becoming a dumping ground.

When to use it? Use the split whenever you add a new value to the app.

```
Local component state -> useState
Form state             -> React Hook Form
Server state           -> TanStack Query
Client UI state        -> Zustand
Shareable filters      -> URL search params
```

Tip

The question is not "What state library should I use?" It is "Who owns this value?"

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

Zustand for Client UI State

What is it? Zustand is a small hook-based client-state store.

Why use it? It is useful for UI state shared across distant components without Redux-style boilerplate.

When to use it? Use it for modals, selected rows, sidebar state, and UI preferences - not server data.

```
type IssueUiStore = {
  selectedIssueId: string | null;
  selectIssue: (issueId: string) => void;
  clearSelection: () => void;
};

export const useIssueUiStore = create<IssueUiStore>((set) => ({
  selectedIssueId: null,
  selectIssue: (issueId) => set({ selectedIssueId: issueId }),
  clearSelection: () => set({ selectedIssueId: null }),
}));
```

Don't

Do not copy API data into Zustand when TanStack Query already owns it.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

TanStack Query for Server State

What is it? TanStack Query manages server data: loading, caching, retries, refetching, and invalidation.

Why use it? Server state is not the same as client UI state. It can become stale and must sync with the backend.

When to use it? Use it for API-loaded resources such as issues, projects, users, and dashboards.

```
export function useIssues(filters: IssueFilters) {  
  return useQuery({  
    queryKey: ["issues", filters],  
    queryFn: () => getIssues(filters),  
    staleTime: 30_000,  
  });  
}
```

Tip

Good query keys describe the data. Include filters in the key when filters change the result.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

Mutations, Invalidation, and Optimistic Updates

What is it? A mutation changes server data. Invalidation tells queries to refetch. Optimistic updates update the UI before the server confirms.

Why use it? These patterns make server updates feel responsive while keeping cache data correct.

When to use it? Use optimistic updates for low-risk reversible changes like status toggles; be careful with destructive actions.

```
const mutation = useMutation({  
  mutationFn: updateIssueStatus,  
  onSuccess: () => {  
    queryClient.invalidateQueries({ queryKey: ["issues"] });  
  },  
});
```


Tip

Optimistic updates duplicate some server logic on the client. Use them when the UX gain is worth the rollback complexity.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

URL State for Filters

What is it? URL state stores shareable values in search parameters.

Why use it? Users can bookmark, share, refresh, and use browser back/forward with the same filters.

When to use it? Use it for search, status filter, sort, and page number in list pages.

```
// /issues?search=login&status=open&page=2
const searchParams = useSearchParams();
const search = searchParams.get("search") ?? "";
const status = searchParams.get("status") ?? "all";
```

Tip

If a filter changes what the page means, it usually belongs in the URL.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

PART 9

Data Tables and Dashboard Pattern

Move from cards to production list screens: tables, metrics, filters, row actions, and bulk workflows.

Table UX Basics

What is it? A table is a dense UI for comparing and acting on records.

Why use it? Issue trackers depend on tables for scanning, filtering, sorting, and bulk work.

When to use it? Use tables when users need to compare many entities, not when they only need a few cards.

```
type IssueRow = {
  id: string;
  title: string;
  status: IssueStatus;
  priority: IssuePriority;
  assigneeName: string | null;
};
```

Tip

Every table needs loading, empty, error, row actions, and a mobile strategy.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

TanStack Table

What is it? TanStack Table is a headless table engine. It gives row models, sorting, filtering, pagination, and selection without forcing UI markup.

Why use it? It handles complex table behavior while letting you use shadcn/ui and Tailwind for presentation.

When to use it? Use it when table behavior grows beyond simple `map` .

```
const columns: ColumnDef<Issue>[] = [
  { accessorKey: "title", header: "Title" },
  { accessorKey: "status", header: "Status" },
  { accessorKey: "priority", header: "Priority" },
];
```

Tip

Headless means you own the HTML and styling. That is powerful, but you must still build accessible table markup.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

Dashboard Metrics and Bulk Actions

What is it? Dashboard metrics summarize the system. Bulk actions let users act on many rows.

Why use it? They turn a list screen into a working operations tool.

When to use it? Use metrics for status overview and bulk actions for repeated workflows.

```
const metrics = {  
  open: issues.filter((i) => i.status === "open").length,  
  blocked: issues.filter((i) => i.status === "blocked").length,  
  done: issues.filter((i) => i.status === "done").length,  
};
```

Tip

Bulk destructive actions should use confirmation dialogs and clear selected counts.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

PART 10

Next.js App Router

Move the learning app into a production React framework with routing, layouts, server/client boundaries, auth, and deployment conventions.

Why Next.js for Production React

What is it? Next.js adds routing, layouts, server rendering, Server Components, route handlers, and deployment conventions.

Why use it? React alone is a UI library; most production apps need a framework around it.

When to use it? Use Next.js when you need routes, nested layouts, auth, server-side data, SEO, and deployment structure.

```
npx create-next-app@latest production-react-app-2026 --ts --app
cd production-react-app-2026
npm run dev
```

Tip

A Vite SPA can be enough for internal apps with a separate backend. Next.js shines when route/data/server concerns belong in the frontend app.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

App Router Structure

What is it? The App Router uses files and folders to define routes, layouts, loading states, and errors.

Why use it? It gives each route a clear place for UI and route-level behavior.

When to use it? Use `app/` for route entries, not for all business code.

```
app/
  (auth)/login/page.tsx
  (dashboard)/layout.tsx
  (dashboard)/issues/page.tsx
  (dashboard)/projects/[projectId]/page.tsx
  api/issues/route.ts
```

Tip

Keep route files thin. Delegate feature UI to `features/*`.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

Server Components vs Client Components

What is it? Server Components run on the server by default. Client Components opt into browser interactivity with `"use client"` .

Why use it? This split lets you keep non-interactive data fetching on the server while using client components for hooks and browser APIs.

When to use it? Use Client Components for state, effects, refs, event handlers, and browser-only APIs.

```
// Server Component by default
export default async function IssuesPage() {
  const projects = await getProjectsForFilter();
  return <IssuesClient initialProjects={projects} />;
}

// Client Component
"use client";
export function IssuesClient() {
  const [search, setSearch] = useState("");
}
```

Tip

Do not add "use client" at the top of everything. Put it only where interactivity is needed.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

Route Handlers and Server Actions

What is it? Route Handlers define API endpoints. Server Actions let forms call server code directly in supported workflows.

Why use it? Both let frontend and backend concerns live near the app when that is the right architecture.

When to use it? Use Route Handlers for REST-like endpoints and integrations. Use Server Actions for form-centric mutations when the trade-offs fit.

```
export async function GET() {
  const issues = await db.issue.findMany();
  return Response.json({ items: issues });
}

export async function POST(request: Request) {
  const payload = await request.json();
  const issue = await createIssue(payload);
  return Response.json(issue, { status: 201 });
}
```

```
continued
```

```
}
```

Apply it to the project. Use this behind TanStack Query when the frontend owns the API surface.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

Authentication, Authorization, and Environment Config

What is it? Authentication identifies the user. Authorization decides what the user can do. Environment config separates secrets from client-visible values.

Why use it? Production apps fail when protected UI is only hidden on the client.

When to use it? Use server-side permission checks for protected routes and mutations.

```
// Client-side hiding is UX, not security
{canDeleteIssue ? <DeleteIssueButton /> : null}

// Server-side check is security
await requirePermission(currentUser, "issue:delete");
```

Tip

Variables prefixed with `NEXT_PUBLIC_` are exposed to the browser. Do not put secrets there.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

PART 11

Common Tools Used in Production React Teams

Use common team tooling without turning the project into dependency soup.

Package Managers and Scripts

What is it? Package managers install dependencies and run project scripts.

Why use it? Scripts make common workflows repeatable for the team and CI.

When to use it? Use clear scripts for dev, build, lint, typecheck, test, and format.

```
{
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "lint": "next lint",
    "typecheck": "tsc --noEmit",
    "test": "vitest",
    "test:e2e": "playwright test",
    "format": "prettier --write ."
  }
}
```

Tip

Pick npm or pnpm and commit the lockfile. Do not mix lockfiles in one project.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

ESLint, Prettier, and TypeScript Strict

What is it? Code quality tools automate formatting and catch common defects.

Why use it? They keep the team from debating style and catch mistakes before review.

When to use it? Run them locally and in CI.

```
{
  "compilerOptions": {
    "strict": true,
    "noUncheckedIndexedAccess": true
  }
}
```

Tip

Strict mode may feel noisy at first, but it forces unclear data boundaries into the open.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

Useful Utility Libraries

What is it? Small utility libraries remove common friction.

Why use it? They are valuable when they reduce repeated boilerplate without hiding business logic.

When to use it? Use `clsx`, `tailwind-merge`, `class-variance-authority`, `date-fns`, `lucide-react`, and `sonner` only where they earn their place.

```
export function cn(...inputs: ClassValue[]) {  
  return twMerge(clsx(inputs));  
}
```

Don't

Do not install a library for every small problem. Extra dependencies become maintenance work.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

PART 12

Backend Integration for a Frontend Engineer

Use your backend experience to design clean frontend/backend boundaries.

Mock API First

What is it? A mock API lets frontend work continue before the real backend is ready.

Why use it? It enables UI development, testing, and contract discussion early.

When to use it? Use mocks for local development, tests, and early prototypes.

```
export const handlers = [  
  http.get("/api/issues", () => {  
    return HttpResponse.json({ items: mockIssues });  
  }),  
];
```

Tip

Mocks should follow the same response shape as the real API. Otherwise they train the frontend to expect the wrong contract.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

DTOs and API Contracts

What is it? A DTO is the shape exchanged between frontend and backend.

Why use it? Contracts prevent accidental mismatch between request payloads, response payloads, and validation rules.

When to use it? Use explicit request and response types for every feature API.

```
export type CreateIssueRequest = {  
  title: string;  
  description?: string;  
  priority: IssuePriority;  
};  
  
export type IssueResponse = {  
  id: string;  
  title: string;  
  status: IssueStatus;  
};
```

Tip

Do not assume your database model is the frontend response model. APIs should return what the client needs, not every internal field.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

FastAPI, C#/.NET, or Next Route Handlers

What is it? The frontend can integrate with a separate backend or use Next.js Route Handlers.

Why use it? Your backend experience is an advantage if you keep the contract clean.

When to use it? Use separate FastAPI/.NET services for larger backend domains; use route handlers for frontend-owned or app-local endpoints.

```
// Frontend client stays the same regardless of backend implementation
export async function getIssues(filters: IssueFilters): Promise<Issue[]> {
  return apiClient.get("/issues", { searchParams: filters });
}
```

Tip

Keep CORS, auth tokens, and error shapes consistent. The frontend should not special-case every endpoint.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

PART 13

Testing and Quality

Test the app as a user would experience it, with mocks and E2E flows where they add confidence.

Testing Strategy

What is it? Testing strategy decides what to test at each level.

Why use it? Different tests catch different problems; one test type cannot do everything.

When to use it? Use Vitest for functions, React Testing Library for components, MSW for API behavior, and Playwright for critical flows.

```
Pure functions      -> Vitest
Components          -> React Testing Library
HTTP behavior       -> MSW
Critical user flow  -> Playwright
```

Tip

Test behavior, not implementation. A user does not care what state variable you used; they care what appears and what actions work.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

Vitest and React Testing Library

What is it? Vitest runs fast unit/component tests. React Testing Library encourages user-centered component tests.

Why use it? Together they cover component behavior without launching a real browser for every test.

When to use it? Use them for forms, empty states, filters, and UI interactions.

```
render(<IssueList issues={} onChange={vi.fn()} />);
expect(screen.getByText(/no issues found/i)).toBeInTheDocument();
```

Tip

Prefer queries by role, label, and text. Those are closer to how users experience the page.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

MSW for API Mocking

What is it? MSW intercepts real HTTP requests in tests and local dev.

Why use it? It avoids mocking fetch everywhere and tests the app closer to production behavior.

When to use it? Use MSW for success, error, empty, and slow API states.

```
export const handlers = [  
  http.get("/api/issues", () => {  
    return HttpResponse.json({ items: mockIssues });  
  }),  
];
```

Tip

A good mock tests the same request path your production code uses.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

Playwright E2E Testing

What is it? E2E tests run real user flows in a browser.

Why use it? They catch routing, auth, forms, network, and browser behavior together.

When to use it? Use them for a few critical flows, not every tiny component.

```
test("user can create an issue", async ({ page }) => {  
  await page.goto("/issues");  
  await page.getByRole("button", { name: /create issue/i }).click();  
  await page.getLabel(/title/i).fill("Fix login loading state");  
  await page.getByRole("button", { name: /save/i }).click();  
  await expect(page.getByText(/fix login loading state/i)).toBeVisible();  
});
```

Tip

Use user-facing locators such as role, label, and text. Avoid brittle CSS selectors for behavior tests.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

PART 14

Production Hardening

Prepare the app to survive real users, production failures, and team maintenance.

Error Handling and Error Boundaries

What is it? Error handling turns failures into understandable UI. Error boundaries catch render-time errors below them.

Why use it? Production apps fail. Good apps fail clearly and recover where possible.

When to use it? Use explicit API errors, form errors, error states, and route-level error boundaries.

```
export default function Error({ error, reset }: ErrorProps) {
  return (
    <ErrorState
      title="Something went wrong"
      description={error.message}
      action={<Button onClick={reset}>Try again</Button>}
    />
  );
}
```

Tip

Expected errors should have product copy. Unexpected errors should be logged and shown safely.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

Observability Basics

What is it? Observability is how you understand what happened in production.

Why use it? Without it, users report bugs before you know anything is wrong.

When to use it? Track errors, failed API calls, important actions, and release versions.

```
captureException(error, {
  tags: { feature: "issues" },
  extra: { issueId },
});
```

Tip

Do not log secrets, tokens, or sensitive form values.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

Performance Techniques

What is it? Performance work reduces unnecessary work and large bundles.

Why use it? React apps can become slow from extra renders, heavy tables, and oversized client bundles.

When to use it? Optimize after measuring, not by guessing.

```
const visibleIssues = useMemo(() => {  
  return issues.filter(matchesFilters);  
}, [issues, matchesFilters]);
```

Tip

Use memoization when it avoids real work. Do not wrap everything in `useMemo` and `useCallback` by default.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

Security Basics

What is it? Frontend security means safe rendering, correct auth assumptions, and careful secret handling.

Why use it? Client code is visible to users. It cannot be trusted for authorization.

When to use it? Use server-side permission checks, safe HTML handling, and private environment variables.

```
// UX: hide button  
{canDeleteIssue ? <DeleteButton /> : null}  
  
// Security: enforce on server  
await requirePermission(user, "issue:delete");
```

Tip

Never trust client-side roles as the only permission check.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

CI/CD and Dependency Maintenance

What is it? CI/CD runs quality checks before deployment. Dependency maintenance keeps the app secure and upgradeable.

Why use it? Human review alone misses repeatable checks.

When to use it? Run lint, typecheck, tests, build, and E2E smoke tests before merge/deploy.

```
name: CI
on: [pull_request]
jobs:
  checks:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci
      - run: npm run lint
      - run: npm run typecheck
      - run: npm test
      - run: npm run build
```

Tip

Keep upgrade notes. Most painful frontend upgrades are painful because nobody recorded local conventions and trade-offs.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

PART 15

Legacy React You Will See

Learn enough legacy React to survive real codebases without copying old patterns into new work.

Class Components

What is it? Class components are older React components that use classes, lifecycle methods, and `this.setState` .

Why use it? Many companies still have them in older codebases.

When to use it? Learn to read them; avoid writing new ones unless maintaining existing code.

```
class IssueCard extends React.Component<Props, State> {  
  componentDidMount() {  
    document.title = "Issue";  
  }  
  
  render() {  
    return <article>{this.props.issue.title}</article>;  
  }  
}
```

Tip

Modern replacement: function components with hooks and clearer data flow.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

Old Redux Boilerplate

What is it? Old Redux uses actions, reducers, selectors, and thunks with a lot of ceremony.

Why use it? You will still see it in large enterprise apps.

When to use it? Recognize it, but prefer TanStack Query for server state and Zustand for small client UI state in this book.

```
dispatch({ type: "issues/statusChanged", payload: { issueId, status } });
```

Tip

Redux Toolkit is still reasonable for complex app state. Old hand-written Redux boilerplate is usually not the first choice for new small apps.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

HOCs, Render Props, and Pages Router

What is it? HOCs and render props were common before hooks. Next.js Pages Router was the previous routing model before App Router.

Why use it? Legacy patterns appear in older libraries and codebases.

When to use it? Learn to recognize them so you can maintain or migrate gradually.

```
// HOC shape
export default withAuth(IssuesPage);

// Render prop shape
<DataProvider render={(data) => <IssueList issues={data} />} />;
```

Tip

Do not rewrite legacy code just to modernize. Migrate when there is a feature, bug, or risk that justifies the change.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

PART 16

Capstone Project

Tie the whole book together into a portfolio-ready production React app.

Capstone Architecture

The capstone is the production-grade Issue Tracker Dashboard. It combines the book into one coherent app.

CAPSTONE ROUTE MAP

```
app/  
  (auth)/login/page.tsx  
  (dashboard)/layout.tsx  
  (dashboard)/dashboard/page.tsx  
  (dashboard)/projects/page.tsx  
  (dashboard)/projects/[projectId]/page.tsx  
  (dashboard)/issues/page.tsx  
api/issues/route.ts  
api/projects/route.ts
```

FEATURE OWNERSHIP

```
features/issues  
  api -> issues-api.ts  
  components -> issue-table, issue-form, filters  
  hooks -> use-issues, use-update-issue  
  schemas -> issue-schema  
  types -> issue-types  
  
features/projects  
features/auth
```

Architecture rule

Pages compose features. Features own business UI. Shared components own generic UI. API files own HTTP.

Build Checklist

What is it? The build checklist turns the capstone into concrete work.

Why use it? It gives you a plan to finish the app without jumping randomly between tools.

When to use it? Follow it in order.

1. Create Next.js project
2. Add Tailwind and shadcn/ui
3. Add folder structure and aliases
4. Build AppShell, Sidebar, TopNav
5. Build issues API layer
6. Add TanStack Query provider
7. Build IssuesTable with filters
8. Add IssueForm with React Hook Form + Zod
9. Add mutations and optimistic status update

continued

10. Add auth and protected routes
11. Add tests
12. Add deployment checklist

Tip

The capstone should be portfolio-quality, not enterprise-perfect. Make trade-offs clear in the README.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

Code Review and Portfolio README

What is it? The final review checks whether the app looks like a professional project. The README explains your decisions.

Why use it? A portfolio project must be understandable without you in the room.

When to use it? Write the README after the app works, but keep notes as you build.

```
# Production React App 2026

## Features
- Dashboard
- Projects
- Issues table
- Create/edit issue forms
- Optimistic status updates
- Role-based UI

## Tech Stack
React, TypeScript, Next.js App Router, Tailwind, shadcn/ui,
TanStack Query, Zustand, React Hook Form, Zod, Vitest, MSW, Playwright.

## Architecture Decisions
Explain why server state uses TanStack Query and client UI state uses Zustand.
```

Tip

Screenshots matter. Include at least dashboard, issues table, create issue dialog, empty state, and test output.

Checkpoint. You can explain where this idea belongs in the Issue Tracker and what file would change.

Reference Basis

This ebook is written against the current official documentation and stable production practices for the tools listed below. Always check project docs before upgrading dependencies.

- React documentation - components, hooks, Thinking in React, Effects, Refs, and current version guidance.
- Next.js App Router documentation - project structure, route conventions, Server and Client Components.
- TanStack Query documentation - queries, mutations, invalidation, and optimistic updates.
- Zustand documentation - hook-based client state.
- Tailwind CSS documentation - utility-first styling.
- shadcn/ui documentation and changelog - open-code components and CLI/registry behavior.
- React Hook Form and resolver documentation - form state and schema integration.
- Zod documentation - TypeScript-first schema validation.
- Vitest, Testing Library, MSW, and Playwright documentation - testing stack.